

Title	奇梦者阵列方案 SDK 接口文档(c)
-------	---------------------

Revision History:

Version	Date	Author	Details of Changes
V1.0.1	2017.2.12	奇梦者	
V1.0.2	2017.3.16	奇梦者	
V1.0.3	2017.7.10	奇梦者	
V.1.0.4	2017.9.12	奇梦者	
V.1.0.5	2019.3.25	奇梦者	Sdk-yk.0.2.0

声明:

本文档是苏州奇梦者网络科技有限公司的财产,奇梦者网络科技有限公司对其内容拥有完全知识产权。该文档仅提供给授权的相关人士作为参考。在未得到苏州奇梦者网络科技有限公司的书面许可之前,任何人不得向其他人、组织透露其中任何细节。如果您不确定您是否属于被授权的人士,请在阅读本文档前联络:rongfei.ma@qdreamer.com。

Disclaim:

This document is the property of Qdreamer. It shall be communicated to authorized personnel only. It is not to be disclosed outside the group without prior written consent. If you are not sure if you are authorized to read the document, please contact rongfei.ma@qdreamer.com before reading.

目 录

1. 概述.....	1
2. SDK 数据结构说明.....	1
2.1 错误码监听等级.....	1
2.2 输入数据结构.....	1
2.3 输出数据结构.....	1
3. SDK Session 模块说明.....	4
3.1 初始化 session.....	4
3.2 退出 session.....	5
4. SDK Engine 模块说明.....	6
4.1 创建引擎.....	6
4.2 设置回调函数.....	8
4.3 启动引擎.....	9
4.4 引擎处理数据.....	9
4.5 重置引擎.....	9
4.6 取消引擎.....	10
4.7 设置引擎.....	10
4.8 销毁引擎.....	11
5. SDK Module 模块说明.....	12
5.1 创建 module.....	12
5.2 设置回调函数.....	12
5.3 启动 module.....	13
5.4 停止 module.....	13
5.5 设置 module.....	13
5.6 销毁 module.....	14
6.Windows 特别说明.....	15

1. 概述

本文档是奇梦者 sdk 接口(C)文档。参考此说明文档，用户可以快速集成搭建自己的 demo, 验证、体验奇梦者的语音服务。

2. SDK 数据结构说明

数据结构中包括输入类型和返回值数据结构说明。

2.1 错误码监听等级

> **qtk_errcode_level_t :**

- 说明：用来指定错误码的监听等级，有 debug，warn，error 三种等级
- 数据结构声明：

```
typedef enum{  
    QTK_DEBUG ,           //debug  
    QTK_WARN ,            //warn  
    QTK_ERROR ,           //error  
}qtk_errcode_level_t;
```

2.2 输入数据结构

> **qtk_feed_type_t :**

- 指定输入数据的是否结束标志位。例如：流式输入音频，QTK_FEED_DATA 表示音频数据还没有结束，当输入最后一帧音频数据将标志位设置成 QTK_FEED_END, 表示音频输入结束。

- 数据结构声明：

```
typedef enum{  
    QTK_FEED_DATA = 0,  
    QTK_FEED_END = 1,  
}qtk_feed_type_t;
```

2.3 输出数据结构

> **qtk_var_type_t :**

● 回调结果的消息类型，如阵列算法引擎(bfio)唤醒消息 QTK_AEC_WAKE，语音识别(asr)引擎会回调 QTK_ASR_TEXT 和 QTK_ASR_HINT 消息，声音检测引擎(vad)会回调 QTK_SPEECH_START 和 QTK_SPEECH_DATA_PCM QTK_SPEECH_DATA_OGG QTK_SPEECH_END 消息。除了每种类型各自表示不同的含义外，总体来说结果类型可分为两种：一种是纯通知消息，如 QTK_SPEECH_START，当接收到该消息时，表示检测说话开始。另外一种携带数据的消息通知，如 QTK_SPEECH_DATA_PCM，当接收到该消息时，表示有算法处理后的音频数据回调，同时有数据上抛。具体属性见 API 注释。

● 数据结构声明：

```
typedef enum{  
    QTK_SPEECH_START,           //检测到说话开始  
    QTK_SPEECH_DATA_OGG,       //回调算法处理后音频数据，OGG 格式  
    QTK_SPEECH_DATA_PCM,       //回调算法处理后音频数据，WAV 格式  
    QTK_SPEECH_END,            //检测到说话结束  
    QTK_AEC_WAKE,              //回调唤醒消息  
    QTK_AEC_DIRECTION,         //回调唤醒方位角  
    QTK_AEC_WAKE_INFO,         //回调唤醒时消息  
    QTK_AEC_SLEEP,             //进入休眠  
    QTK_AEC_CANCEL,            //cancel 消息通知  
    QTK_AEC_CANCEL_DATA,       //cancel 的数据  
    QTK_AEC_WAKE_ONESHOT,      //ONESHOT 模式唤醒  
    QTK_AEC_WAKE_NORMAL,       //普通唤醒  
    QTK_AEC_THETA_HINT,        //回调可能声源角度  
    QTK_AEC_THETA_BF_BG,       //回调算法抑制的声源角度  
    QTK_AEC_THETA_BG,          //回调背景角度  
    QTK_TTS_START,             //开始语音合成  
    QTK_TTS_DATA,              //语音合成数据  
    QTK_TTS_END,               //语音合成结束  
    QTK_ASR_TEXT,              //回调识别结果  
    QTK_ASR_HINT,              //回调识别中间结果  
    QTK_EVAL_TEXT,             //回调评测结果  
    QTK_SEMDLG_TEXT,           //回调语义结果  
    QTK_VAR_ERR,               //回调运行时错误码  
    QTK_VAR_SOURCE_AUDIO,      //回调一路原始音频数据  
    QTK_AUDIO_LEFT,            //回调连接信息，audio 设备拔出  
    QTK_AUDIO_ARRIVE,          //回调连接信息，audio 设备已连接  
    QTK_AUDIO_ERROR,           //audio 设备错误，需要重置  
}qtk_var_type_t;
```

> **qtk_var_t :**

- 回调结果，该结构包括回调结果类型和回调结果实体数据
- 数据结构声明：

```
typedef struct
{
    qtk_var_type_t type;
    union{
        double f;
        int i;
        short s;
        struct{
            int theta;
            int phi;
        }ii;
        struct{
            char *data;
            int len;
        }str;
        struct{
            float theta;
            int on;
        }fi;
    }v;
}qtk_var_t;
```

3. SDK Session 模块说明

Session 模块提供用户验证、服务器分配和 log 日志等功能，是 SDK 各功能模块使用都必须初始化的模块。

3.1 初始化 session

➤ 函数原型：

```
qtk_session_t* qtk_session_init(char *params, qtk_errcode_level_t *level,  
void *ths, qtk_errcode_handler *errhandler);
```

➤ 函数功能：

Session 初始化，执行用户验证、服务器分配、获取错误码和日志功能初始化，包括设置错误码显示级别和设置错误码结果回调函数。

➤ 参数：

- **params**：指定 Session 初始化参数，为键值对的方式
- **level**：设置错误码显示等级
- **ths**：回调实例指针
- **errhandler**：回调函数指针，回调函数类型为 qtk_errcode_handler

➤ 参数 params 说明：

- **cache_path**：**String** 类型，指定缓存目录，缓存内容包括 log 日志、dns 缓存、验证信息缓存
- **public_key**：**String** 类型，指定公钥（由奇梦者提供，不提供的话默认即可）
- **phrase**：**String** 类型，指定为“qdreamer”，默认即可
- **log**：**String** 类型，指定 log 路径，默认生成 log 路径为 \${cache_path}/qvoice.log
- **dns**：**String** 类型，指定 dns 缓存路径，默认生成 dns 缓存路径为 \${cache_path}/qvoice.dns
- **auth**：**String** 类型，指定 auth 缓存路径，默认生成 auth 缓存路径为 \${cache_path}/qvoice.auth
- **appid**：**String** 类型，用户 appid，必须指定
- **secretkey**：**String** 类型，用户 secretkey，必须指定
- **host**：**String** 类型，指定服务器 host，指定后自动设置 use_srvsel=0
- **port**：**String** 类型，指定服务器 port，默认为 80
- **timeout**：**Int** 类型，指定服务器超时时间

- `use_srvsel` : **BOOL** 类型，是否动态服务器分配
- `use_auth_cache` : **BOOL** 类型，是否缓存验证信息
- `use_log` : **BOOL** 类型，是否保存日志信息
- `use_timer` : **BOOL** 类型，是否使用定时器，默认 0
- `use_cldhub` : **String** 类型，是否使用网络客户端，默认使用，可指定关闭

3.2 退出 session

➤ 函数原型：

```
void qtk_session_exit(qtk_session_t *session);
```

➤ 函数功能：

退出 Session

➤ 参数：

`session` : 实例指针

➤ 返回值：无

4. SDK Engine 模块说明

engine 模块封装的是单独的功能模块，例如阵列算法引擎，识别引擎等。目前支持的功能模块有：

- (1) 语音识别 (asr)
- (2) 连续语音识别 (csr)
- (3) 阵列算法 (bfio)
- (4) 语音评测 (eval)
- (5) 语义对话 (semdlg)
- (6) 语音合成 (tts)
- (7) 语音检测 (vad)
- (8) 唤醒 (wakeup)

4.1 创建引擎

➤ 函数原型：

```
qtk_engine_t* qtk_engine_new(qtk_session_t *session, char *params);
```

➤ 函数功能：

创建引擎实例

➤ 参数：

- **params**：实例指针

➤ 参数说明：

(1) 通用模块：

- **role**：**String** 类型，指定引擎类型，取值【asr,csr,bfio,eval,semdlg,tts,vad,wakeup】，必须指定且只指定其一；
- **use_bin**:**BOOL** 类型，指定配置文件类型，默认为 True (1)，使用 bin 格式；
- **cfg**：**String** 类型，指定配置文件路径，必须指定；
- **use_thread**：**BOOL** 类型，是否使用内置线程，默认为 True (1)，使用内置线程；
- **winStep**：**Int** 类型，当输入音频时，设置音频输入每一帧的时长，单位毫秒 (ms)。
- **output_ogg**：**Bool** 类型，输出是否设置 OGG 格式

- **syn** : **Bool** 类型, 线程间是否同步
- **timealive** : **int** 类型, 设置定时器 timer 的时间长度, -1 为不设置
- **use_onemic** : **Bool** 类型, 是否使用单麦
- **asr_min_time** : **Int** 类型, 语音识别的最小识别间隔
- **asr_max_time** : **Int** 类型, 语音识别的最大识别间隔

(2) 识别模块(asr):

- **rec_min_conf** : **Float** 类型, 指定最小通用识别置信度, 默认为 1.9, 推荐(1.5--2.5);
- **usrec_min_conf** : **Float** 类型, 指定最小语法识别置信度, 默认为 2.0, 推荐(1.5--2.5);
- **grammar_min_conf** : **Float** 类型, 指定最小用户定制识别置信度, 默认为 2.0, 推荐(1.5--2.5);
- **use_json** : **BOOL** 类型, 设置识别结果是否以 json 串的格式返回, 默认返回 jaon 串格式;
- **ebnf_fn** : **String** 类型, 设置语法识别网络初始化文件路径, 设置后替代内置的语法网络。

(3) 连续识别模块(csr):

- **rec_min_conf** : **Float** 类型, 指定最小通用识别置信度, 默认为 1.9, 推荐(1.5--2.5);
- **usrec_min_conf** : **Float** 类型, 指定最小语法识别置信度, 默认为 2.0, 推荐(1.5--2.5);
- **grammar_min_conf** : **Float** 类型, 指定最小用户定制识别置信度, 默认为 2.0, 推荐(1.5--2.5);
- **use_json** : **BOOL** 类型, 设置识别结果是否以 json 串的格式返回, 默认返回 jaon 串格式;
- **left_margin** : **Int** 类型, 设置内置 VAD 的左缓冲区帧数, 推荐值 20
- **right_margin** : **Int** 类型, 设置内置 VAD 的右缓冲区帧数, 推荐值 20
- **ebnf_fn** : **String** 类型, 设置语法识别网络初始化文件路径, 设置后替代内置的语法网络。

(4) 阵列算法模块(bfio):

- **mic** : **Int** 类型, 设置麦克风阵列的 mic 数量

- **mic%d** : **ARRAY** 类型, 设置第%d 个麦克风的坐标信息[x,y,z]
- **wakewrd** : **String** 类型, 设置通用唤醒词, 当资源为通用唤醒时可以设置

(5) 对话模块(semdlg):

- **idle_time** : **Int** 类型, 设置多长时间无对话引擎进入休眠
- **idle** : **Bool** 类型, 设置是否使用休眠
- **active** : **Bool** 类型, 设置是否使用启动自动唤醒

(6) 合成模块(tts):

- **res** : **String** 类型, 指定合成音角色。当识别引擎为云端识别时设置才生效
- **tts_volume** : **Float** 类型, 指定合成音音量, 默认值 1.0, 推荐值(0--10.0]
- **tts_speed** : **Float** 类型, 指定合成音语速, 默认值 1.0, 推荐值(0--3.0]
- **tts_pitch** : **Float** 类型, 指定合成音语调, 默认值 1.0, 推荐值(0--3.0]

(7) 语音检测模块(vad):

- **left_margin** : **Int** 类型, 设置 VAD 的左缓冲区帧数。推荐值 20
- **right_margin** : **Int** 类型, 设置 VAD 的右缓冲区帧数。推荐值 20

(8) 唤醒模块(wake):

- **wake_logwav** : **Bool** 类型, 设置是否缓存唤醒音频数据

4.2 设置回调函数

> 函数原型:

```
int qtk_engine_set_notify(qtk_engine_t *e, void *ths, qtk_engine_notify_f  
notify_f);
```

> 函数功能:

设置结果回调函数

> 参数:

- **e** :引擎实例指针
- **ths** : 回调实例指针
- **notify_f** : 回调函数指针, 类型为 **qtk_engine_notify_f**

> 返回值:

- 0 : 成功
- -1 : 失败

4.3 启动引擎

➤ 函数原型：

```
int qtk_engine_start(qtk_engine_t *e);
```

➤ 函数功能：

启动引擎实例

➤ 参数：

- **e** :引擎实例指针

➤ 返回值：

- 0 : 成功
- -1 : 失败

4.4 引擎处理数据

➤ 函数原型：

```
int qtk_engine_feed(qtk_engine_t *e, char *data, int bytes, qtk_feed_type_t is_end);
```

➤ 函数功能：

输入运行数据，支持音频流式输入。如识别 asr 模块，输入标志位 is_end 设置为 QTK_FEED_DATA 表示输入还没结束，当输入结束，将标志位设置为 QTK_FEED_END

➤ 参数：

- **e** :引擎实例指针
- **data** : 输入数据
- **bytes** : 输入数据长度
- **is_end** : 输入标志位

➤ 返回值：

- 0 : 成功
- -1 : 失败

4.5 重置引擎

➤ 函数原型：

```
int qtk_engine_reset(qtk_engine_t *e);
```

➤ 函数功能：

重置引擎，与启动引擎配对使用。每次启动引擎，完成操作后必须要重置引擎才能进行下一次操作

➤ 参数：

- **e** :引擎实例指针

➤ 返回值：

- 0：成功
- -1：失败

4.6 取消引擎

➤ 函数原型：

```
int qtk_engine_cancel(qtk_engine_t *e);
```

➤ 函数功能：

取消掉前面所有的输入处理，目前该接口只有 asr 模块支持

➤ 参数：

- **e** :引擎实例指针

➤ 返回值：

- 0：成功
- -1：失败

4.7 设置引擎

➤ 函数原型：

```
int qtk_engine_set(qtk_engine_t *e, char *params);
```

➤ 函数功能：

设置 Engine 运行时参数，键值对形式

➤ 参数：

- **e** :引擎实例指针

- **params** : 运行时参数，键值对，各模块支持参数如下：

(1) 语音识别(asr):

- **ebnf** :**String** 类型，设置用户定制识别网络，下一次识别生效一次

(2) 阵列算法(bfio):

- **wakewrd** :**String** 类型，设置通用唤醒词，需在 start 之前设置

- **mic** :**String** 类型，设置 mic 信息，需在 start 之前设置

- **beam** :**float** 类型，设置人相对阵列的角度，start 之后设置

(2) 语音评测(eval):

- **evaltxt** :**String** 类型，设置评测文本

(3) 语音合成(tts):

- **tts_volume** :设置合成音音量，推荐值【0-10.0】

- **tts_speed**:设置合成音语速，推荐值【0-3.0】

- `tts_pitch`: 设置合成音语调, 推荐值【0-3.0】

4.8 销毁引擎

- 函数原型:

```
int qtk_engine_delete(qtk_engine_t *e);
```

- 函数功能:

销毁 Engine 实例

- 参数:

- `e`: 引擎实例指针

- 返回值:

- 0: 销毁成功
- -1: 销毁失败

5. SDK Module 模块说明

module 模块提供基于 engine 的集成功能模块。例如阵列识别模块 (bfasr) 集成了阵列算法引擎,识别引擎和录音机的功能,用户插上阵列麦克风执行就能够得到识别结果输出。

目前 module 支持的功能有:

- (1) asr: 录音机+识别引擎
- (2) bfio: 录音机+阵列算法引擎
- (3) eval: 录音机+评测引擎
- (4) wakeup : 录音机+唤醒引擎

5.1 创建 module

> 函数原型:

```
qtk_module_t* qtk_module_new(qtk_session_t *session, char *res, char *cfg_fn);
```

> 函数功能:

创建 module 实例

> 参数:

- session :session 实例指针
- res :指定 module 功能模块,字符串类型,取值【asr,bfio,eval,wakeup】
- cfg_fn:指定 module 资源路径,字符串类型

> 返回值:

- 成功:返回实例指针
- 失败:返回 NULL

5.2 设置回调函数

> 函数原型:

```
int qtk_module_set_notify(qtk_module_t *m, void *user_data, qtk_engine_notify_f notify);
```

> 函数功能:

设置结果回调函数

> 参数:

- m :module 实例指针
- user_data :回调实例指针

- **notify** : 回调函数指针,回调函数类型为 `gtk_engine_notify_f`

➤ 返回值 :

- 成功 : 返回 0
- 失败 : 返回 -1

5.3 启动 module

➤ 函数原型 :

```
int gtk_module_start(gtk_module_t *m);
```

➤ 函数功能 :

启动 module

➤ 参数 :

- **m** :module 实例指针

➤ 返回值 :

- 成功 : 返回 0
- 失败 : 返回 -1

5.4 停止 module

➤ 函数原型 :

```
int gtk_module_stop(gtk_module_t *m);
```

➤ 函数功能 :

停止 module

➤ 参数 :

- **m** :module 实例指针

➤ 返回值 :

- 成功 : 返回 0
- 失败 : 返回 -1

5.5 设置 module

➤ 函数原型 :

```
int gtk_module_set(gtk_module_t *m,char *params);
```

➤ 函数功能 :

设置 module 运行时参数

➤ 参数 :

- **m** :module 实例指针
- **params** : 键值对形式, 各模块支持设置的参数和 engine 相同

- 返回值：
 - 成功：返回 0
 - 失败：返回 -1

5.6 销毁 module

- 函数原型：

```
void qtk_module_delete(qtk_module_t *m);
```
- 函数功能：

消除 module
- 参数：
 - `m`: module 实例指针
- 返回值：
 - 成功：返回 0
 - 失败：返回 -1

6.Windows 特别说明

sdk 默认采用的编码方式为 UTF-8，因此在 windows 系统中使用本文档所述 API 时，可能会出现输入文件（cfg 配置、语义设置等）和输出文件（日志等）不同程度的乱码。建议将输入文件转换为 UTF-8 格式载入，将输出文件转换为 GBK 格式使用。